

UNIT V: Multi-Threading

1. What is a thread?

A: A thread is a single, lightweight path of execution within a process. A process can have multiple threads running concurrently.

2. What is the difference between multitasking and multithreading?

A: Multitasking is when a CPU runs multiple processes (like Chrome, Word, Spotify) at the same time. Multithreading is when a single process (like Chrome) runs multiple threads (like one tab, another tab, a download) at the same time.

3. What are the two ways to create a thread in Java?

A:

1. By **extending** the `Thread` class.
2. By **implementing** the `Runnable` interface.

4. Which method must you override to define the thread's task?

A: The public void `run()` method. This method contains the code that the thread will execute.

5. How do you start a thread?

A: You call the `start()` method.

6. What happens if you call the `run()` method directly instead of `start()`?

A: If you call `run()` directly, no new thread is created. The `run()` method will just execute on the current thread, like a normal method call.

7. Which way of creating a thread is better, and why?

A: Implementing `Runnable` is generally better. Java does not allow extending multiple classes, so if you extend `Thread`, you cannot extend any other class. By implementing `Runnable`, your class is free to extend another class.

8. What are the main states in a thread's life cycle?

A:

1. **New:** The thread object is created.
 2. **Runnable:** The thread is ready to run (after `start()` is called).
 3. **Running:** The thread scheduler has selected the thread to run.
 4. **Blocked (Waiting):** The thread is temporarily inactive (e.g., waiting for I/O or a lock).
 5. **Terminated:** The thread's `run()` method has finished.
9. How can a thread enter the Blocked state?
- A: By calling methods like `sleep()`, `wait()`, or by waiting for an I/O operation to complete.
10. What is a thread priority?
- A: It's a number (from 1 to 10) that suggests to the thread scheduler how important a thread is.
11. What are the three standard priority constants?
- A:
- `Thread.MIN_PRIORITY` (value 1)
 - `Thread.NORM_PRIORITY` (value 5)
 - `Thread.MAX_PRIORITY` (value 10)
12. What is a "race condition"?
- A: A race condition is an error that occurs when multiple threads try to access and modify the same shared resource at the same time, leading to an unpredictable and incorrect result.
13. What is thread synchronization?
- A: Synchronization is the process of controlling when threads can access a shared resource. It ensures that only one thread can use the resource at a time, preventing race conditions.
14. What is the keyword used to achieve synchronization?
- A: The `synchronized` keyword. It can be used on a method or as a block of code.

15. What is a "monitor" or "lock"?

A: A monitor (or lock) is an object-level lock that a thread must acquire before it can enter a synchronized method or block. No other thread can enter until the first thread releases the lock.

16. What is inter-thread communication?

A: It's the mechanism that allows synchronized threads to coordinate with each other. For example, one thread (a "producer") can tell another thread (a "consumer") that data is ready.

17. Which three methods are used for inter-thread communication?

A: wait(), notify(), and notifyAll().

18. What does the wait() method do?

A: It tells the current thread to release the lock it holds and go into a waiting state.

19. What does the notify() method do?

A: It wakes up a single thread that is currently waiting on the same object's lock.

20. Where must wait(), notify(), and notifyAll() be called from?

A: They must be called from within a synchronized block or method. If not, Java will throw an `IllegalMonitorStateException`.